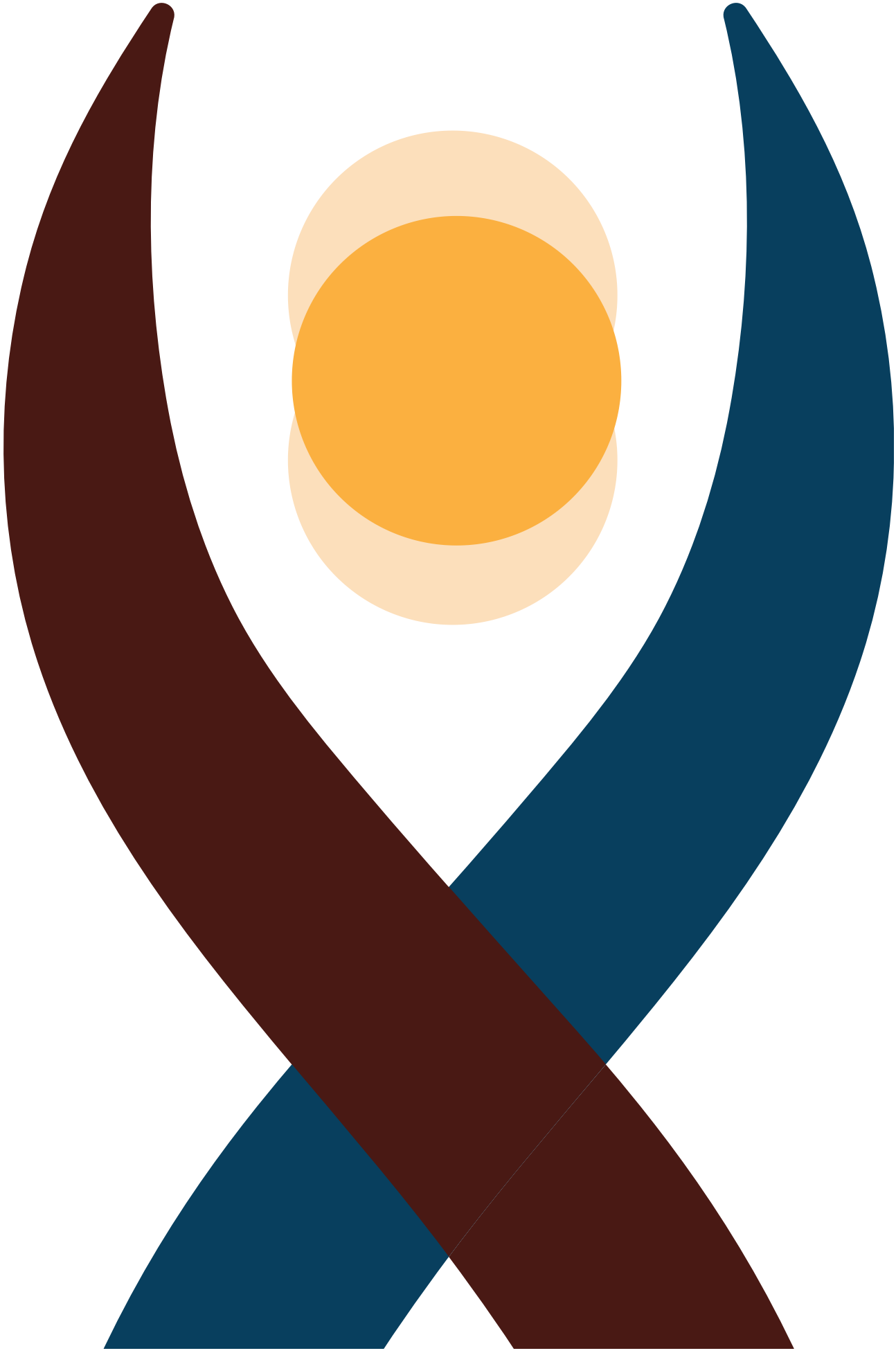
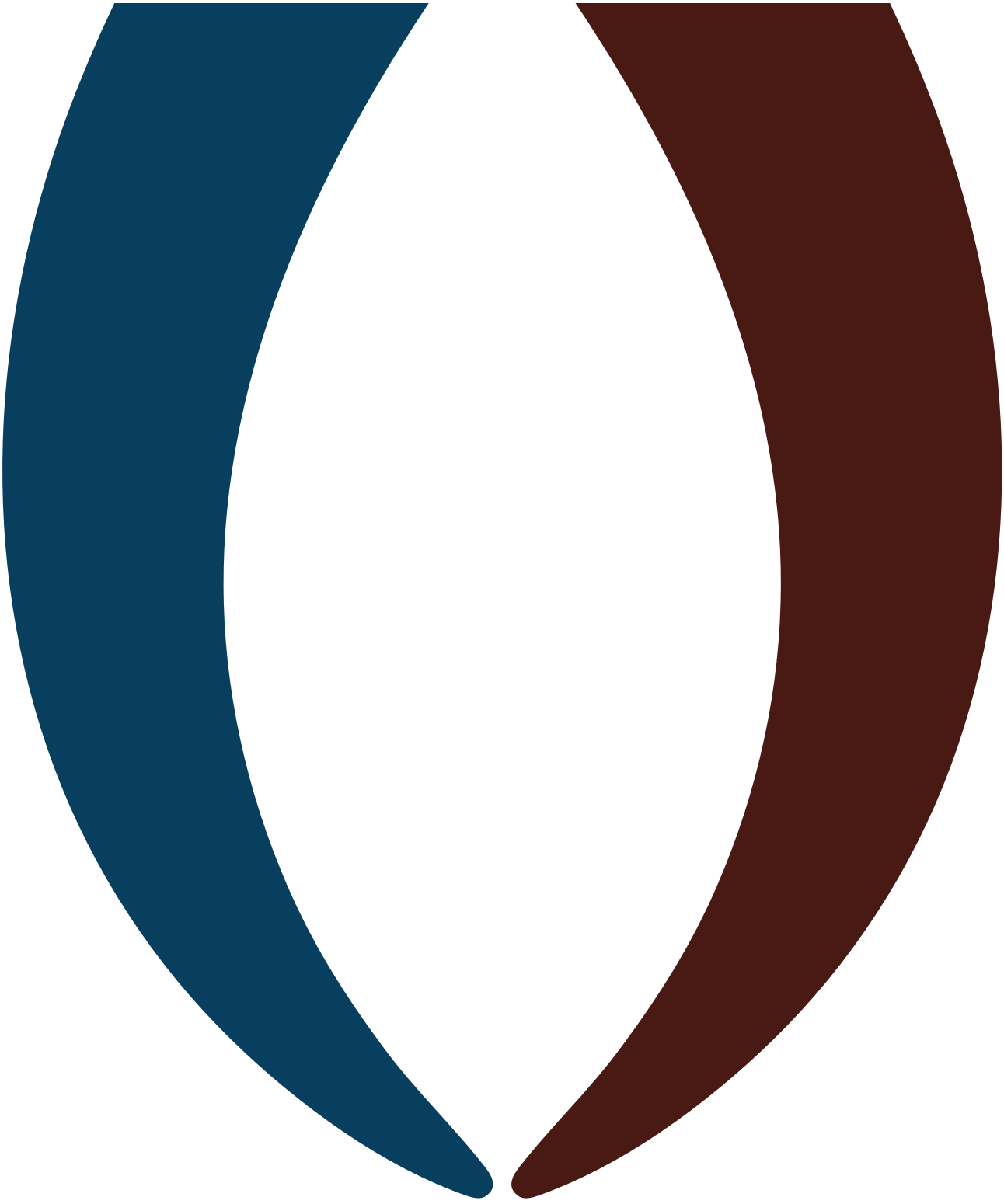
---

title: BioViL Energy — Project Development & Architecture Log author: Antigravity AI Assistant date: March 27, 2026 version: 1.0.0 classification: Confidential Internal Documentation

---





BioViL Energy: Comprehensive Architecture & Development Log

> [!IMPORTANT] **Executive Definition** This document archives the exact technical trajectory, version control states, design methodologies, and architectural decisions made over the

> course of developing the **BioViL Energy KPI Tracking System and Automated Corporate Reporting Suite**.

🏗️ Part 1: Architecture & Technical Foundations

1.1 The Web Dashboard (`index.html`, `script.js`, `style.css`)

We completely overhauled the dashboard to represent a premium, globally competitive brand rather than just a basic data interface.

- **Design Framework (BioViL CI):** Implemented a rigorous "Glassmorphism" aesthetic.
  - *Primary Color:* Deep Teal (`#083f5e`)
  - *Accent Color:* BioViL Orange (`#f7941d`)
  - *Success Color:* Emerald (`#10b981`)
  - *Typography:* Google **Outfit** (Weights 300, 400, 600, 800) for a modern, tech-forward feel.

- **Routing & Responsiveness:**
  - Built a custom single-page-application (SPA) hash-router (`#project1` → `project-view`), seamlessly switching between tabs without page reloads.
  - Engineered a responsive sidebar that intelligently collapses into icon-only mode below 1200px width.
- **Dynamic Data Engine:**
  - Anchored calculations to a hard `START_DATE` of Jan 15, 2026.
  - Calculates `weeksPassed` live on page load, automatically pushing cumulative data to the cards (CO_2e avoidance, Firewood reduction, etc.) without manual entry.

1.2 The Biological Visualization Model (Chart.js)

Instead of static, linear charts drawn from basic arrays, we built a fully reactive biological simulator for real-time dashboard plotting:

1. **Mesophilic Curve Calibration:** Hardcoded a full 12-month biological mapping specific to Assam's climate, accurately depressing winter yields to `0.016 m³/kg` (psychrophilic stalling) and maximizing summer peaks up to `0.035 m³/kg` (mesophilic optimum).
2. **Deterministic Noise Injection:** Wrote a `seededRandom()` function to apply `±15%` weekly variance to waste input, simulating realistic human behavioral fluctuations in dung collection while keeping the charts reproducible per page load.

Part 2: Automated Corporate Reporting Engine

A critical milestone was abandoning manually captured browser screenshots in favor of an automated, Python-powered HTML-to-PDF compiler capable of producing investor-ready collateral.

2.1 The PDF Build Script (`build_corporate_pdfs.py`)

This script acts as the heart of your document rendering pipeline.

- **HTML Structure & Compilation:** Generates native HTML structures bound to identical CSS used in the dashboard for exact brand parity.
- **The Headless Chrome Engine:** To achieve pixel-perfect print rendering, we bypassed weak PDF libraries (like `weasyprint` or `pdfkit`) in favor of triggering your local system's **Google Chrome Headless Print API**.

> [!TIP] **CSS Print Layout Triumph** Overcoming Chrome's pagination limitations was difficult. `position: fixed` headers overlap body text during pagination. We engineered an elite workaround: By locking the entire document inside an HTML `<table>` wrapper and using `<thead>` and `<tfoot>` spacing blocks matching the exact pixel heights of the global header/footer (`65px`/`45px`), Chrome is forced to reserve that physical block of A4 paper on *every subsequent page break*, achieving perfect page flow.

2.2 Reusable Template Generator (`generate_docx_template.py`)

Because you requested native editable functionality in the same CI:

- Written utilizing `python-docx`.
- Programmatically sets exact A4 centimeters (`21.0 x 29.7`), precise RGB hex injections (`#083F5E`, `#f7941d`), and sets up hierarchical Styles (H1, H2, Title, Normal) directly in the `.docx` metadata.
- **XML Hardening:** Solved MS Word file corruption bugs by ensuring dynamic `.docx` elements (like footer page numbers) were strictly written inside proper text `<w:r>` (Run) elements instead of raw `<w:p>` (Paragraph) nodes.

📈 Part 3: Version Control & Iteration Log

Below is the chronological sequence of our codebase evolution from concept to current state:

v0.5.0: The Static Sandbox

- Imported raw project data, initial manual creation of dashboard layout.
- *Limitation:* Hardcoded arrays (`labels: ['Week 1', 'Week 2']`); no auto-progression. Dashboard aesthetics felt dated.

v1.0.0: The Dynamic Rebuild

- *Action:* Complete UI rewrite into Deep Teal & Glassmorphism.
- *Action:* JavaScript logic overhauled. Built the `weeksPassed` time-engine and `seededRandom()` noise generator.
- *Result:* Dashboard became a living document anchored to real-world time.

v1.1.0: First Draft Markdown Docs

- *Action:* Extracted IPCC/EPA calculations into `BioViL_Detailed_Report.md`.
- *Action:* Generated the first set of Python charts (`yield_curve.png`, etc.) using `matplotlib` to replace dashboard screenshots.

v2.0.0: The Premium Render Launch

- *Action:* Python generation suite created. Shifted documentation from Markdown files into the `build_corporate_pdfs.py` compiler.
- *Action:* Designed the `100vw/100vh` full-bleed gradient cover page, pulling CSS elements behind the print margins for a borderless edge.

v2.1.0 (Current): Final Polish & CI Architecture

- *Bugfix:* Detailed Report header overlapping on Pages 4, 6, 8, 10 resolved via targeted DOM `<table>` wrapping.

- *Enrichment:* One-Pager converted to a 3-column data-dense grid housing all primary graphical assets (CO_2, Yield, Budget).
- *Bugfix:* DOCX raw XML corruption sorted.

📂 Part 4: File Tree Anatomy (Current Output Directory)

```
Dashboard_Draft/Documentation/
├── build_corporate_pdfs.py           # Core HTML generation protocol
├── generate_docx_template.py         # MS Word native schema builder
├── BioViL_One_Pager.html             # Intermediary render stage
├── BioViL_Detailed_Report.html       # Intermediary render stage
├── BioViL_One_Pager.pdf              # FINAL: Executive Handshake Document
├── BioViL_Detailed_Report.pdf        # FINAL: 5-Page Technical Run-Book
├── BioViL_Report_Template.docx       # FINAL: Reusable CI Asset Form
├── BioViL_KPI_Calculation_Model.xlsx # FINAL: Raw Spreadsheet Sandbox
├── yield_curve.png                   # Python compiled asset
├── co2_impact.png                    # Python compiled asset
├── co2_cumulative.png                # Python compiled asset
└── budget_chart.png                  # Python compiled asset
```

> [!NOTE] **Future Action Plan** The current system is 100% complete for Prototype Phase 1 reporting and pitching to stakeholders. The immediate next requirement will likely be replacing the deterministic `seededRandom()` logic with active API endpoints when your IoT sensors (temperature/flow) go live.